Why the Forward-Sensitivity Recursion Is Correct

T6 explanatory note

11 September 2026

Short answer

The recursion is the multivariable chain rule applied to one simulator step. It is not an integration formula. The next state depends on the control in two ways: directly through the step function, and indirectly because the current state already depends on all earlier effects of the control. The two terms in the recursion represent exactly these two paths.

1 Begin with the simulator, not its derivative

Let $g \in \mathbb{R}^p$ be the fixed control vector and $X_k \in \mathbb{R}^n$ the complete continuous simulator state at step k. One forward step is

$$X_{k+1} = F_k(X_k, g).$$

Although X_k is written as an argument, it is itself a function of g after the simulation has begun:

$$X_{k+1}(g) = F_k\big(X_k(g), g\big).$$

Define the sensitivity matrix

$$S_k = \frac{dX_k}{dg} \in \mathbb{R}^{n\times p}.$$

Entry (i, j) says how state component i at time k changes under a small change in control component j.

2 Differentiate the composed step

Apply the chain rule to $F_k(X_k(g), g)$:

$$\begin{aligned} S_{k+1} &= \frac{d}{dg} F_k\big(X_k(g), g\big) \\ &= \underbrace{\frac{\partial F_k}{\partial X_k}}_{A_k} \underbrace{\frac{dX_k}{dg}}_{S_k} + \underbrace{\frac{\partial F_k}{\partial g}}_{B_k}. \end{aligned}$$

Therefore

$$\boxed{S_{k+1} = A_k S_k + B_k.}$$

The meanings are:

Term	Meaning
B_k	Direct same-step effect of changing g while pretending the incoming state X_k is fixed.
S_k	Sensitivity already stored in the incoming state because g affected earlier steps.
$A_k S_k$	Propagation of that inherited state sensitivity through the current transition.
S_{k+1}	Total sensitivity of the next state, containing both the inherited and new direct effects.

This resolves the apparent paradox. B_k alone is only a partial derivative; it is not the complete gradient path from F_k to g. The sum $A_k S_k + B_k$ is the total derivative of the composed mapping $g \mapsto F_k(X_k(g), g)$.

3 Why this is not an integration back to the transition function

The recursion computes a derivative from known local Jacobians and the previous sensitivity. No integration with respect to g occurs. If one later integrated $S_{k+1}(g)$ with respect to g, one could reconstruct $X_{k+1}(g)$ only up to a control-independent constant and only with suitable information over the integration path. That inverse operation is unrelated to how the recursion is derived.

Also, F_k is a transition function of two arguments, while $X_{k+1}(g)$ is the result of composing F_k with the control-dependent state $X_k(g)$. Differentiating the composition yields S_{k+1}; it does not assert that a derivative equals F_k.

4 A scalar example

Consider

$$x_{k+1} = ax_k + bg,$$

where x_0 is independent of g. Here

$$A_k = \frac{\partial F}{\partial x} = a, \qquad B_k = \frac{\partial F}{\partial g} = b, \qquad S_0 = 0.$$

The recursion gives

$$S_1 = b, \qquad S_2 = aS_1 + b = ab + b, \qquad S_3 = a^2b + ab + b.$$

Directly expanding the state gives

$$x_2 = a^2x_0 + (ab + b)g,$$

so

$$\frac{dx_2}{dg} = ab + b = S_2.$$

The b term is the direct effect introduced at step 1; the ab term is the earlier effect, first introduced at step 0 and then propagated through the state by a. Omitting A_kS_k would forget the entire simulation history.

5 Traffic interpretation

Suppose g changes effective signal service. At step k it can affect traffic through two routes:

1. **Direct route:** g changes current movement capacity and therefore current discharge. This is part of B_k.
2. **Inherited route:** earlier changes in discharge altered current queues, lane occupancy, bay space, ordered residual mass, and upstream spillback. The sensitivity of those states is S_k, and the current traffic equations propagate it through A_kS_k.

For the finite-bay case, an earlier increase in left-turn service can empty the bay, increase its next-step receiving space, admit more left-turn mass, reduce feeder blockage, and change later total system time. That multi-step path lives in repeated multiplication by the state Jacobians. The recursion unfolds as

$$S_K = B_{K-1} + A_{K-1}B_{K-2} + A_{K-1}A_{K-2}B_{K-3} + \cdots,$$

showing that every earlier direct control effect is transported through all subsequent state transitions.

6 From state sensitivity to objective sensitivity

If the objective is

$$J(g) = \sum_{k=0}^{K-1} \ell_k(X_k),$$

then another application of the chain rule gives

$$\nabla_g J = \sum_{k=0}^{K-1} S_k^\top \nabla_{X_k} \ell_k.$$

The factor S_k says how the control changes the state, and $\nabla_{X_k} \ell_k$ says how that state changes the objective. If ℓ_k also depends directly on g, an additional term $\partial \ell_k / \partial g$ must be added. The proposed total-system-time integrand has no such direct term: g affects it through traffic state.

7 Initial state and nonsmooth cases

The proposal uses $S_0 = 0$ because the initial traffic state is fixed independently of the green allocation. If initialization depended on g, the correct start would instead be $S_0 = dX_0/dg$.

The equations hold as ordinary derivatives within a stable active-set, prefix, and branch signature. At a tie or event boundary, a min, capacity activation, or ordered-prefix change may make the classical derivative undefined or one-sided. The project therefore checks the recursion away from those boundaries and reports boundary behavior separately rather than smoothing it silently.

Takeaway

Forward sensitivity runs a second state alongside the traffic state. The traffic recursion advances "what the system is," while the sensitivity recursion advances "how that system would change under a small change in g." The formula $S_{k+1} = A_k S_k + B_k$ is exactly the chain rule needed to preserve both accumulated history and the new direct effect at each step.